

THE UNIVERSITY OF
MELBOURNE

Final Report

The Art of Scientific Computation Power Laws and Finite Size Scaling in the BTW Abelian Sandpiles Model

Author:
Jonathan Purcell

Subject Handbook code:
COMP90072

Faculty of Science
The University of Melbourne

Draft Version: 9th October 2026

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Mathematical Formulation of the BTW Model . . . 2

2 Simulation **4**
2.1 Method . . . 4
2.2 Practicalities . . . 4
2.3 Results . . . 5

3 Power Law Fit **6**
3.1 Naïve Power Law Fit . . . 6
3.2 Manually-truncated Power Law . . . 9

4 Finite Size Scaling Effect **11**
4.1 Exponentially Truncated Power Law . . . 11

5 Conclusion **14**
5.1 Reflection . . . 14

Bibliography **14**

1

Introduction

1.1 Motivation

1. Introduce the BTW sandpiles model as a model for self-organised criticality and 1/f noise [1]
2. Define criticality, self-organised criticality, power law scaling, etc

1.2 Mathematical Formulation of the BTW Model

The Abelian Sandpiles Model (or BTW Sandpile Model) was introduced by Bak, Tang, and Wiesenfeld in 1987 [2] as an example of a dynamical system driving itself towards criticality, exhibiting what they call "self-organised criticality".

We introduce the sandpile model, adapting the notation of Járai, 2018 [5].

Define a two-dimensional square lattice with lattice size L as the a graph $G_L = (V_L, E_L)$, where

$$V_L := \{(x, y) \in \mathbb{Z}^2 : 1 \leq x, y \leq L\}$$

defines the set of vertices and

$$E_L := \{\mathbf{xy} : \mathbf{x}, \mathbf{y} \in V_L \wedge \|\mathbf{x} - \mathbf{y}\|_1 = 1\}$$

defines the set of edges, where $\|\mathbf{x} - \mathbf{y}\|_1$ denotes the L_1 norm, i.e. $\|(x_1, x_2) - (y_1, y_2)\|_1 = |x_1 - y_1| + |x_2 - y_2|$.

Let $N(\mathbf{x}) = \{\mathbf{y} \in V_L : \mathbf{xy} \in E_L\}$ be the neighbourhood of $\mathbf{x}$ in G.

A sandpile is a map $\eta : V_L \to \mathbb{Z}_{\geq 0}$, and $\eta(\mathbf{x})$ is called the height at site $\mathbf{x}$.

Define the critical height as $\eta_c := \max_{\mathbf{x} \in V_L} \deg_G(\mathbf{x})$. In the above setup, $\eta_c = 4$ provided $L \geq 3$. A site $\mathbf{x} \in V_L$ is called stable iff $\eta(\mathbf{x}) < \eta_c$ and is otherwise unstable. The sandpile η is stable iff it is stable at each site $\mathbf{x} \in V_L$.

Let $U_\eta = \{\mathbf{x} \in V_L : \eta(\mathbf{x}) \geq \eta_c\}$ be the set of unstable sites.

Let $\Psi_{G_L} = \mathbb{Z}_{\geq 0}^{V_L}$ be the set of possible sandpiles and let $\Omega_{G_L} = \{0, 1, 2, ..., \eta_c - 1\}^{V_L} \subseteq \Psi_{G_L}$ be the set of stable sandpiles.

A toppling is a function $\phi : \Psi_{G_L} \times V_L \to \Psi_{G_L}$ mapping $(\eta, \mathbf{x}) \mapsto \eta'$ where

$$\eta'(\mathbf{y}) = \begin{cases} \eta(\mathbf{y}) + 1 & \mathbf{y} \in N(\mathbf{x}) \wedge \mathbf{x} \in U_\eta \\ \eta(\mathbf{y}) - \eta_c & \mathbf{y} = \mathbf{x} \in U_\eta \\ \eta(\mathbf{y}) & \text{otherwise} \end{cases}$$

Note that we allow open boundary conditions in this study. When a toppling is initiated at a boundary site, we have $\ell(\eta, \mathbf{x}) := \sum_{\mathbf{x} \in V_L} \eta(\mathbf{x}) - \sum_{\mathbf{x} \in V_L} \eta'(\mathbf{x}) > 0$. This is called the loss associated with toppling η at $\mathbf{x}$.

The avalanche (stabilising) function $\xi : \Psi_{G_L} \to \Omega_{G_L}$ maps $\eta = \eta^{(0)} \mapsto \eta^{(s)}$ where $\eta^{(s)}$ is the stable sandpile output by the following algorithm:

$i \leftarrow 0$
while $U_{\eta^{(i)}} \neq \emptyset$ **do**:

choose $\mathbf{x}^{(i+1)} \in U_{\eta^{(i)}}$
$\eta^{(i+1)} \leftarrow \phi(\eta^{(i)}, \mathbf{x}^{(i+1)})$
$i \leftarrow i+1$
end while
return $\eta^{(i)} = \eta^{(s)}$

Avalanches can be described by a range of properties:

1. The avalanche (toppling) size is the final index $s(\eta) = s$
2. The avalanche (toppling) area $a(\eta)$ is the size of the set $\{\mathbf{x}^{(i)} : i \in \{1, \cdots, s\}\}$
3. The loss is $\ell(\eta) := \sum_{i=1}^{s} \ell(\eta^{(i-1)}, \mathbf{x}^{(i)})$
4. The L_1 length is $d(\eta) := \max_{i \in \{1,\cdots,s\}} \left\|\mathbf{x}^{(1)} - \mathbf{x}^{(i)}\right\|_1$, with $d(\eta) := 0$ if $s = 0$.

Note the constraints $0 \leq a(\eta) \leq s(\eta)$, $a(\eta) \leq L^2$, $d(\eta) \leq 2L$.

A proof that ξ is well-defined and independent of the choice of $\mathbf{x}^{(j)}$ in the loop can be found in Járai, 2018 [5]. The abelian nature of the topplings were first shown by Dhar in 1990 [4].

1.2.1 Probabilistic Framework

Define the deposition function $\delta : \Psi_{G_L} \times V_L \to \Psi_{G_L}$ as the map $\eta \mapsto \eta'$ with rule

$$\eta'(\mathbf{y}) = \begin{cases} \eta(\mathbf{y}) + 1 & \mathbf{y} = \mathbf{x} \\ \eta(\mathbf{y}) & \mathbf{y} \neq \mathbf{x} \end{cases}$$

We describe the sandpile evolution process by considering the sequence of iid random variables $\mathbf{X}_1, \mathbf{X}_2$..., each with support V_L and pmf

$$p(\mathbf{X}_j = \mathbf{x}) = L^{-2}$$

This represents a sequence of uniformly chosen random vertices, which are taken as locations for grain deposition.

From the sequence $\{X_j\}_{j \geq 1}$, we can build a sequence of stable sandpiles $\{\eta_j\}_{j \geq 0}$ with $\eta_0 = \mathbf{0}$ and $\eta_j = \xi(\delta(\eta_{j-1}, \mathbf{X}_j))$ for $j \geq 1$.

Likewise, we obtain the sequence of avalanche sizes $\{S_j\}_{j \geq 1}$ where $S_j := s(\delta(\eta_{j-1}, \mathbf{X}_j))$. A similar construction yields sequences of avalanche areas, losses, and L_1 lengths.

The sequence $\{\eta_j\}_{j \geq 0}$ forms a discrete Markov Chain with finite state space Ω_{G_L}; the distribution of η_{j+1} depends only on the previous state η_j and the transition probability is determined by the distribution of $\mathbf{X}_j$.

Jarai [5] discusses the Markov structure in detail, and proves that the distribution of states η_j converges to a stationary distribution $\pi(\eta)$ as $j \to \infty$, where $\pi(\eta) > 0$ iff η is a recurrent state in the Markov Chain.

Consequently, the sequence of avalanche sizes $\{S_j\}_{j \geq 1}$ and the other avalanche statistic sequences, which are functions of the states η_j, also converge in distribution to stationary distributions as $j \to \infty$.

1.2.2 Statistical Models

Describe modelling choices for power law and truncated power law distributions

2

Simulation

- Describe the simulation procedure including implementation details (numba, burn-in)
- Show plots of basic results found
- Describe the qualitative scaling behaviour observed across different L

2.1 Method

The stationarity of sequences $\{S_j\}_{j\geq 1}$ and $\{A_j\}_{j\geq 1}$ forms the theoretical foundation for empirical simulation and analysis according to the following algorithm:

initialise $\mathbf{s}, \mathbf{a}$ as empty arrays of size n
initialise $\eta = \mathbf{0}$
for $i \in \{1, \cdots, m\}$ **do**:
simulate $\mathbf{X}_i$
$\eta \leftarrow \delta(\eta, \mathbf{X}_i)$
$\eta \leftarrow \xi(\eta)$
end for
for $i \in \{m+1, \cdots, m+n\}$ **do**:
simulate $\mathbf{X}_i$
$\eta \leftarrow \delta(\eta, \mathbf{X}_i)$
$s[i-m] \leftarrow s(\eta)$
$a[i-m] \leftarrow a(\eta)$
$\eta \leftarrow \xi(\eta)$
end for
return $\mathbf{s}, \mathbf{a}$

In the above, simulated states $\eta_1, \cdots, \eta_m$ are called burn-in states, and are discarded. After this period, we assume that the distribution of η_j is approximately stationary. Therefore, simulations $\eta_{m+1}, \cdots, \eta_{m+n}$ are retained and their empirical distribution approximates the stationary distribution $\pi(\eta)$. From that sequence, we construct simulated sequences S_j and A_j of length n, whose distribution approximates $p(S)$ and $p(A)$ respectively.

2.2 Practicalities

In practice, hundreds of thousands of simulations are required to approach stationarity. This is computationally expensive, especially for large lattice sizes, because computations cannot be parallelised due to the Markov structure of the problem. An initial Python implementation of the simulation procedure above was too slow to produce meaningful results. The final implementation uses `numba` with `njit` decorators, which stands for 'no python just in time' [6]. This compiles the Python code prior to execution, bypassing the CPython interpreter and achieving execution speed near code written in C. However, only a subset of Python and `numpy` functionality is permitted, requiring significant refactoring away from object-oriented design.

2.3 Results

Simulations were performed for lattice sizes $L \in \{5, 10, 20, 30, 40, 50, 65, 80, 100\}$. The burn-in period was set to $m = 4 \times 10^4$ and sequences of length $n = 1 \times 10^6$ were retained for each lattice size.

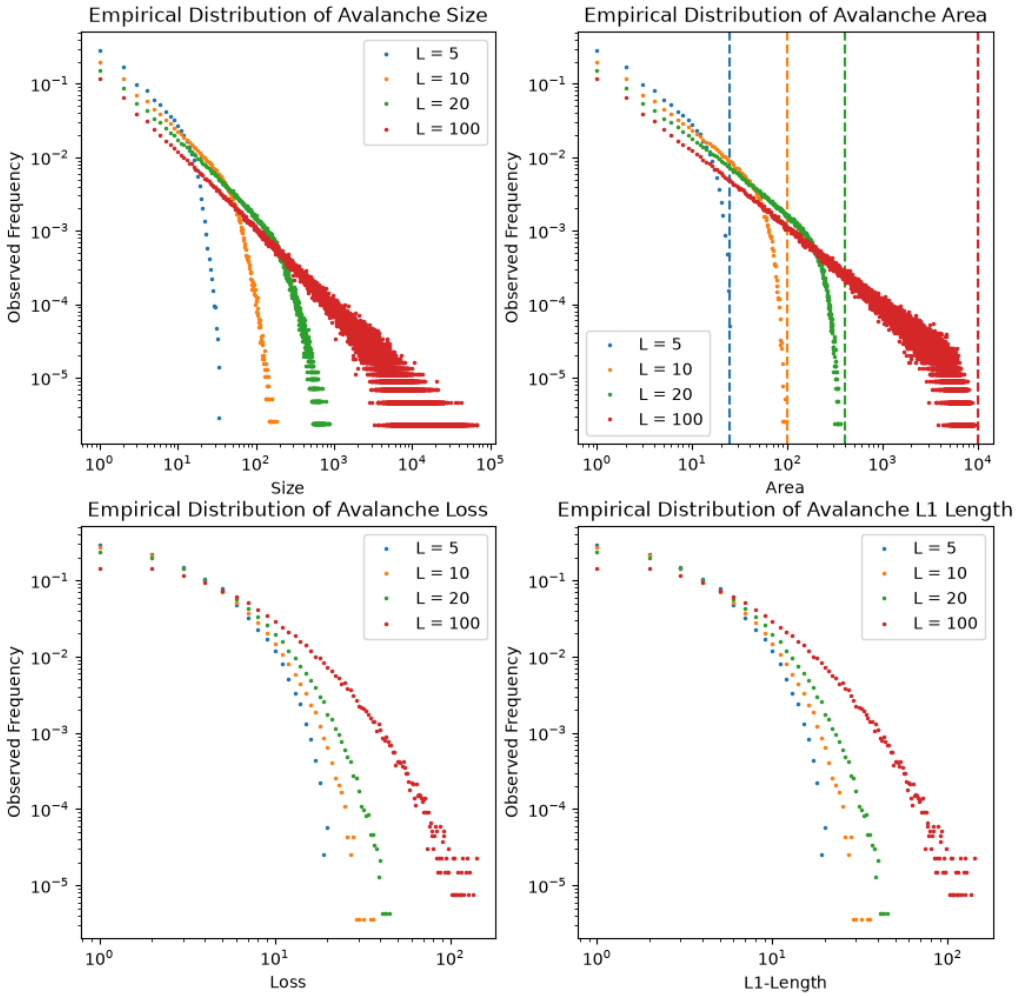


Figure 2.1: Observed frequencies of key avalanche statistics computed from the simulated sandpile Markov chain, excluding the discarded burn-in samples. A subset of four lattice sizes are depicted: 5, 10, 20, and 100, with varying colors. Values are computed from 10^6 simulated observations. The dashed vertical lines on the top right correspond to the maximum possible avalanche area for each lattice size, L^2. Plots are shown in log-log axes. The appearance of horizontal lines is an artifact of the log-log axes. The lines correspond to sizes with only 1, 2, 3, etc. observations in the dataset, with exaggerated vertical distance between the lines because of the log scale.

The plots in Figure 2.1 depect a linear trend in early values of avalanche size and avalanche area, suggesting a power law distribution followed by a sharp decay. The distributions for Loss and L1 Length do not appear to have this power law structure, and the remainder of this report focusses on understanding and analysing the distribution of avalanche size and area.

3

Power Law Fit

1. Naive Power Law Fit
2. Hard Truncated Power Law Fit
3. (Optional) Note on Linear Regression

3.1 Naïve Power Law Fit

We hypothesize a power law scaling relationship for Toppling Size S and area A of the form:

$$p(S = s; \tau, s_{max}) \propto s^{-\tau_s} p(A = a; \tau) \qquad \propto a^{-\tau_a}$$

In the above, s_{max} and a_{max} are known constants.

Let $\mathbf{s} = (s_1, \cdots, s_T)^T, \mathbf{a} = (a_1, \cdots, a_T)^T$ be vectors representing the simulation data for topple size and area respectively.

The likelihood function for topple size (and similarly area) is:

$$\mathcal{L}(\mathbf{s}; \tau_s, s_{max}) = \prod_{i=1}^{T} \frac{s_i^{-\tau_s}}{C_{\tau_s, s_{max}}}$$

$$\log \mathcal{L}(\mathbf{s}; \tau_s, s_{max}) = -T \log C_{\tau_s, s_{max}} - \tau_s \sum_{i=1}^{T} \log s_i$$

Where $C_{\tau_s, s_{max}}$ is the normalising constant:

$$C_{\tau_s, s_{max}} = \sum_{k=1}^{s_{max}} k^{-\tau_s}$$

To estimate the parameters τ_s and τ_a, we compute their Maximum Likelihood Estimates (MLEs). These are the values $\hat{\tau_s}$ and $\hat{\tau_a}$ that satisfy

$$\hat{\tau_s} = \arg\max_{\tau_s} \mathcal{L}(\mathbf{s}; \tau_s, s_{max}) = \arg\min_{\tau_s} \{-\log \mathcal{L}(\mathbf{s}; \tau_s, s_{max})\}$$

and similarly for $\hat{\tau_a}$.

Because the normalising constant $C_{\tau_s, s_{max}}$ depends on the parameter τ_s, we cannot find an analytic solution and rely on numerical optimisation techniques. The numerical optimisation technique used here is the L-BFGS-B algorithm introduced by Byrd et al. (1995) [3] and implemented in the `scipy` python library.

Initially, we ignore the non-linear tails of the distribution and model the power law as though it is valid for all plausible values. For area, there is a natural upper bound $a_{max} = L^2$. For topple size, which is in theory unbounded, we choose an arbitrary upper bound $s_{max} = 2L^2$ which exceeded all observed topple sizes to compute the normalising constant.

The estimates obtained are presented in Figure 3.1 and Table 3.1.

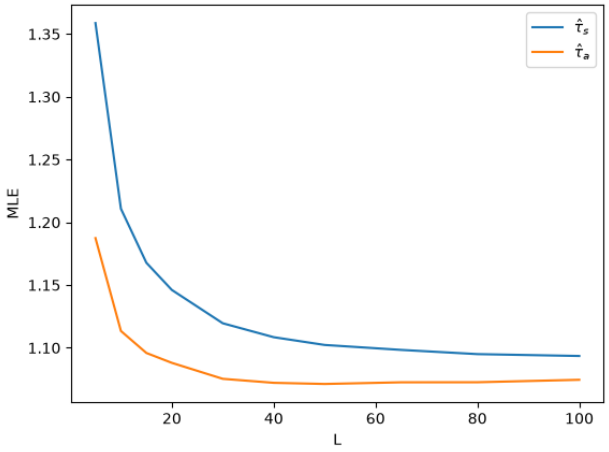


Figure 3.1: Estimates MLEs for τ_s, τ_a for varying lattice sizes L.

L	τ_s	τ_a
5	1.36	1.17
10	1.22	1.12
15	1.17	1.09
20	1.14	1.09
30	1.12	1.07
40	1.11	1.07
50	1.10	1.07
65	1.10	1.07
80	1.09	1.07
100	1.09	1.08

Table 3.1: MLE power-law slopes obtained for varying lattice sizes L

However, inspection of the fit against data in Figure 3.2 indicates that the estimated slope has been overestimated due to the non-powerlaw behaviour of the system at the tails. This effect appears to be most prominent in smaller lattice sizes, but still visibly impacts the fit for $L = 100$.

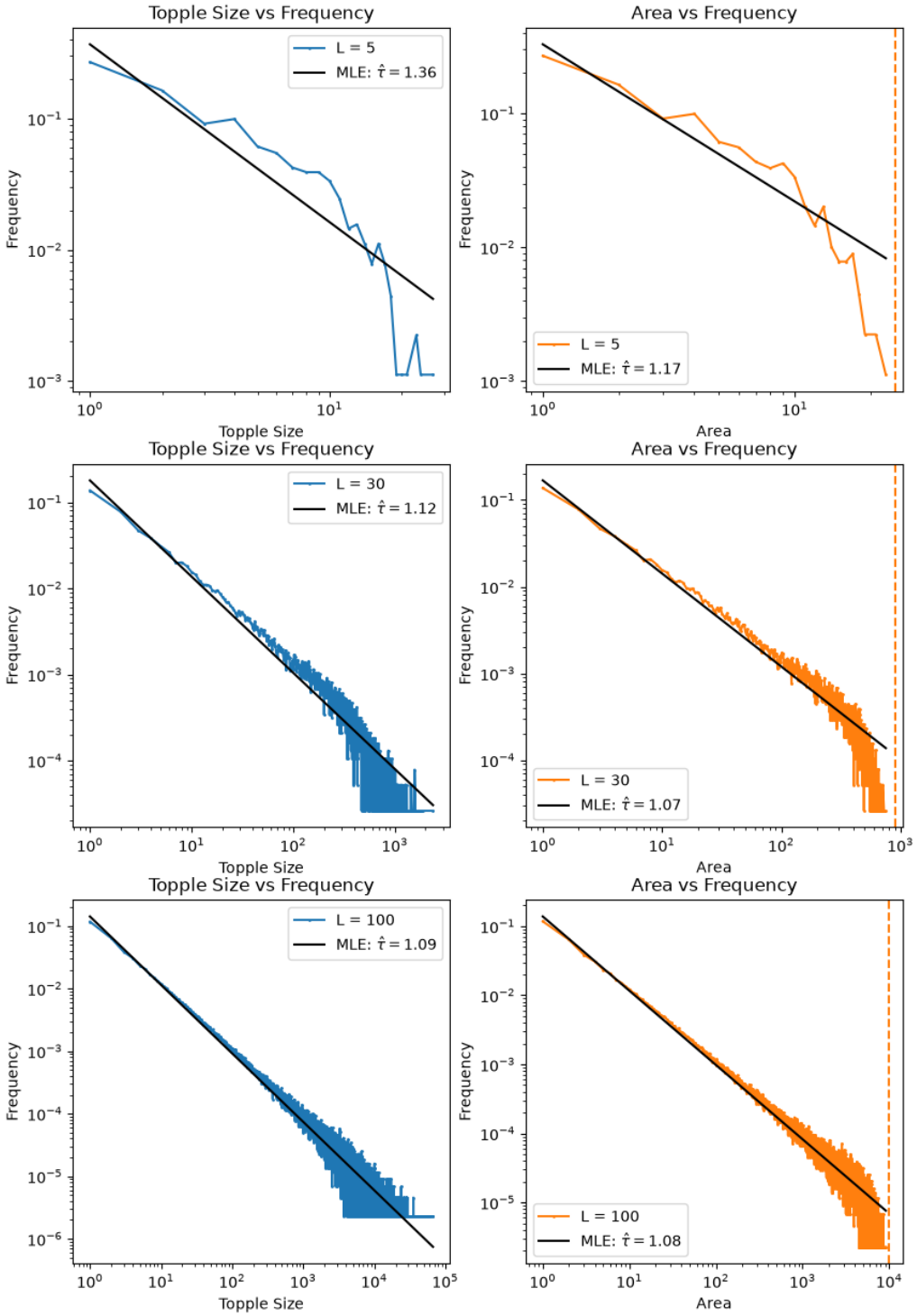


Figure 3.2: Fitted MLEs for a power law distribution against the observed data.

3.2 Manually-truncated Power Law

As a first correction for the finite lattice effect, we consider restricting s_{max} and a_{max} to include only those topple sizes and areas for which linearity is observed in the log-log plot. To avoid manually setting an arbitrary maximum for each value of L, here we consider the rule-of-thumb approximation $s_{max} = a_{max} = 0.1L^2$. Clearly, there are significant limitations to this approach: the cutoff rule was chosen arbitrarily, and data is significantly limited for lower values of L. However this will help us get a more accurate idea of the power law structure. The results in Table 3.2 and Figure 3.3 are noticeably lower than the previous estimates in Table 3.1, with an increasing trend rather than decreasing. From these two analyses, it is reasonable that the powerlaw-governed section of the distribution has exponent $\tau_s, \tau_a \approx 1$, which is consistent with previous papers [1].

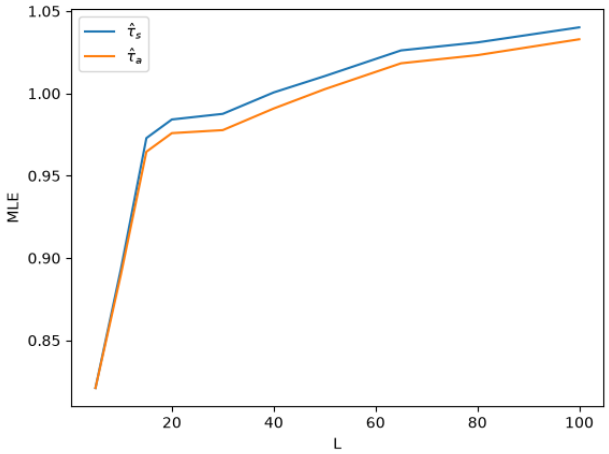


Figure 3.3: Estimates MLEs for τ_s, τ_a for varying lattice sizes L.

L	τ_s	τ_a
5	0.82	0.82
10	0.89	0.89
15	0.97	0.96
20	0.98	0.98
30	0.99	0.98
40	1.00	0.99
50	1.01	1.00
65	1.03	1.02
80	1.03	1.02
100	1.04	1.03

Table 3.2: MLE power-law slopes obtained for varying lattice sizes L under the truncation rule $s_{max} = a_{max} = L^2/10$

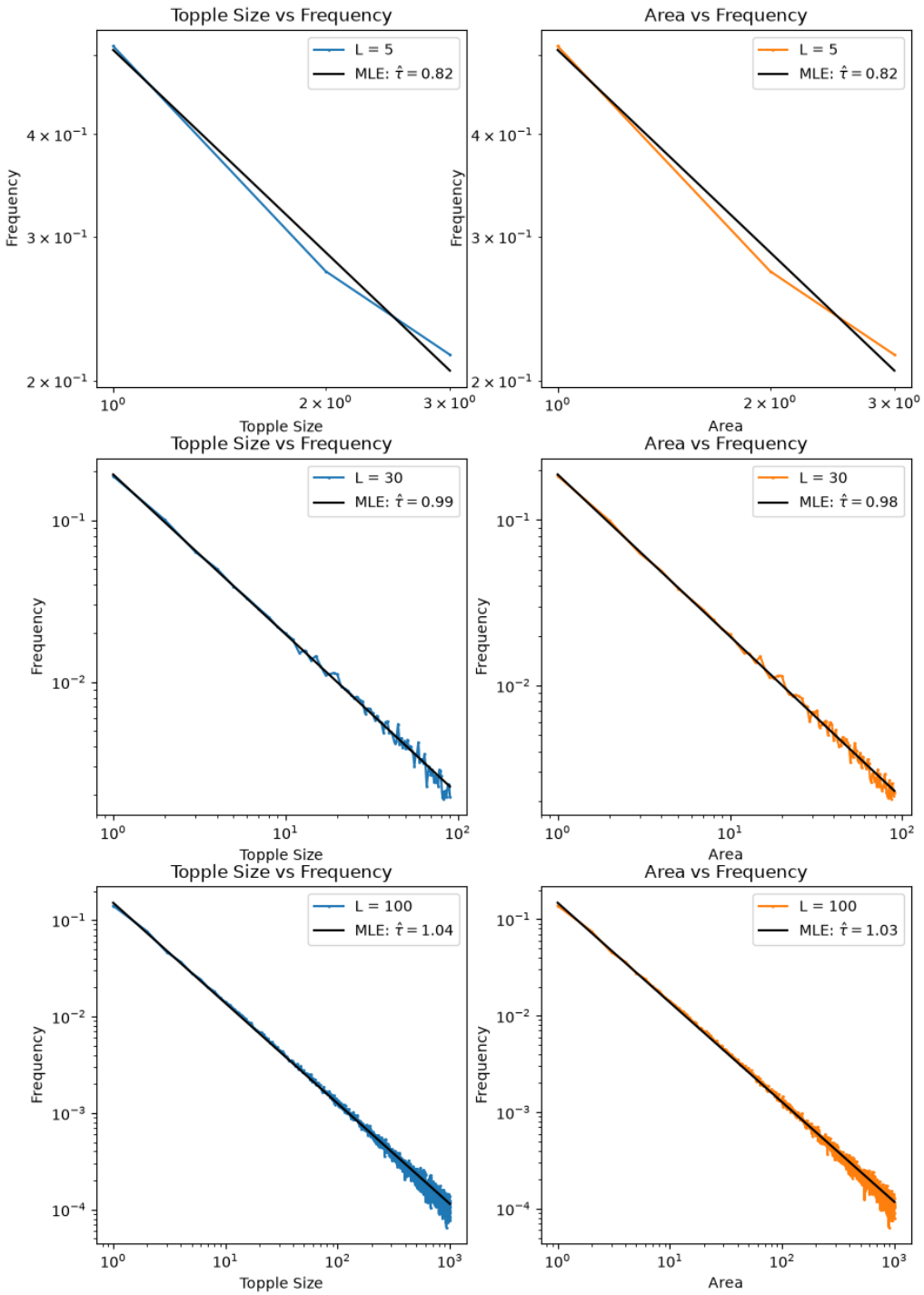


Figure 3.4: Fitted MLEs for a power law distribution against the observed data.

4

Finite Size Scaling Effect

1. General form
2. MLE fit for exponentially-truncated power law
3. MLE fit for generalised exp-truncated power law
4. Discussion

The finite size scaling law hypothesizes that the tail behaviour relates to the finite lattice size L via:

$$p(A = a; \tau_a, L, f, D) \propto a^{-\tau_a} f\left(\frac{a}{L^D}\right)$$

Here f governs the tail behaviour of the distribution, i.e. for small values of u, $f(u) \approx 1$ and the distribution resembles a power law.

4.1 Exponentially Truncated Power Law

We first consider the exponentially-truncated power law with $a_{max} = L^2$ as the natural cutoff. Again, we impose $s_{max} = 2L^2$ for computational convenience.

$$p(A = a; \tau_a, c_a, a_{max}) = \frac{1}{C_{\tau_a, c_a, a_{max}}} a^{-\tau_a} \exp\left(\frac{-a}{c_a}\right)$$

Where $C_{\tau_a, c_a, a_{max}}$ is the normalising constant:

$$C_{\tau_a, c_a, a_{max}} = \sum_{k=1}^{a_{max}} k^{-\tau_a} \exp\left(-k/c_a\right)$$

The likelihood and log-likelihood are:

$$\mathcal{L}(\mathbf{a}; \tau_a, c_a, a_{max}) = \prod_{i=1}^{T} \frac{1}{C_{\tau_a, c_a, a_{max}}} a_i^{-\tau_a} \exp\left(\frac{-a_i}{c_a}\right)$$

$$\log \mathcal{L}(\mathbf{a}; \tau_a, c_a, a_{max}) = -T \log C_{\tau_a, c_a, a_{max}} - \tau_a \sum_{i=1}^{T} \log a_i - \frac{1}{c_a} \sum_{i=1}^{T} a_i$$

In the above, we model $f(u) = \exp(-u)$. We expect $c_a \propto L^D$ for some constant D across all lattice sizes. The advantage of this structure is that the full (not truncated) dataset can be used to fit the powerlaw exponent τ_a, with c_a representing a soft cutoff boundary that is fit to the data, rather than a hard cutoff that is chosen arbitrarily. If the distribution obeys finite size scaling with this f, then we should expect to see a relatively

constant τ_a across varying L, and we should observe a linear $\log(c_a)$ vs $\log(L)$ plot with constant slope D_a. In particular, we can estimate pooled MLEs from the full dataset across different values of L according to

$$p(a; L, \tau_a, \gamma_a, D_a) = \frac{1}{C_{L,\tau_a,\gamma_a,D_a}} a^{-\tau_a} \exp\left(\frac{a}{\gamma_a L^{D_a}}\right)$$

Denote $\mathbf{a}_L$ the set of observations generated from the lattice of size L. Then likelihood and log-likelihood in the pooled case are:

$$\mathcal{L}(\mathbf{a}_{L_1}, \cdots \mathbf{a}_{L_K}; \tau_a, \gamma_a, D_a) = \prod_{j=1}^{K} \mathcal{L}(\mathbf{a}_{L_j}; \tau_a, c_a = \gamma_a L_j^{D_a}, a_{max} = L^2)$$

We follow the below procedure:

1. For each L, estimate MLE parameters $\hat{\tau}_a(L), \hat{c}_a(L)$.
2. Examine the MLE fit against the observed data
3. Pool the data and estimate MLE parameters $\tau_a, gam\hat{i}ma_a, \hat{D}_a$.
4. Examine the MLE fit of $\log \hat{\gamma}_a, \hat{D}_a$ against $\log \hat{c}_a(L)$ and $\log L$.

4.1.1 Estimating MLE parameters for each L

As before, for each L, we numerically optimise the log-likelihood function to obtain:

$$\hat{\tau}_a(L), \hat{c}_a(L) = \underset{\tau_a, c_a}{\arg\min} \left\{ -\log \mathcal{L}(\mathbf{a}_L; \tau_a, c_a, a_{max} = L^2) \right\}$$

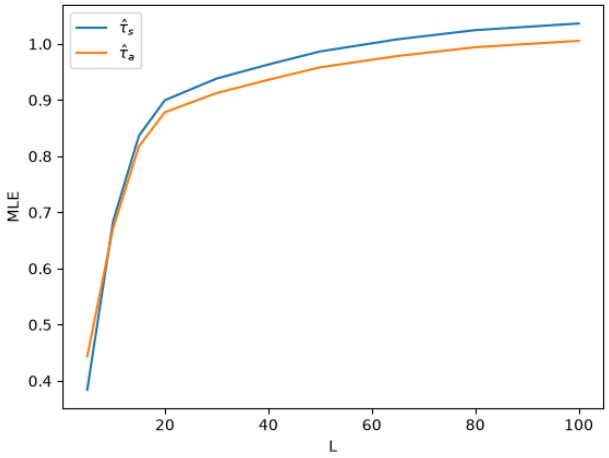


Figure 4.1: Estimates MLEs for τ_s, τ_a for varying lattice sizes L.

L	τ_s	c_s	τ_a	c_a
5	0.38	5.69	0.44	6.27
10	0.68	27.49	0.67	26.58
15	0.84	81.55	0.82	74.76
20	0.90	169.69	0.88	148.70
30	0.94	444.13	0.91	355.15
40	0.96	902.92	0.94	671.42
50	0.99	1623.38	0.96	1124.59
65	1.01	3237.96	0.98	2020.08
80	1.02	5846.02	0.99	3306.74
100	1.04	10599.48	1.01	5372.85

Table 4.1: MLE power-law slopes τ and soft cutoffs c obtained for varying lattice sizes L

The results in Table 4.1 are promising: for lattice sizes 40 and above, the powerlaw coefficient $\tau \approx 1$. Further, the soft cutoffs c appear to scale appropriately with L, a relationship that we will examine in more detail when computing the pooled estimators.

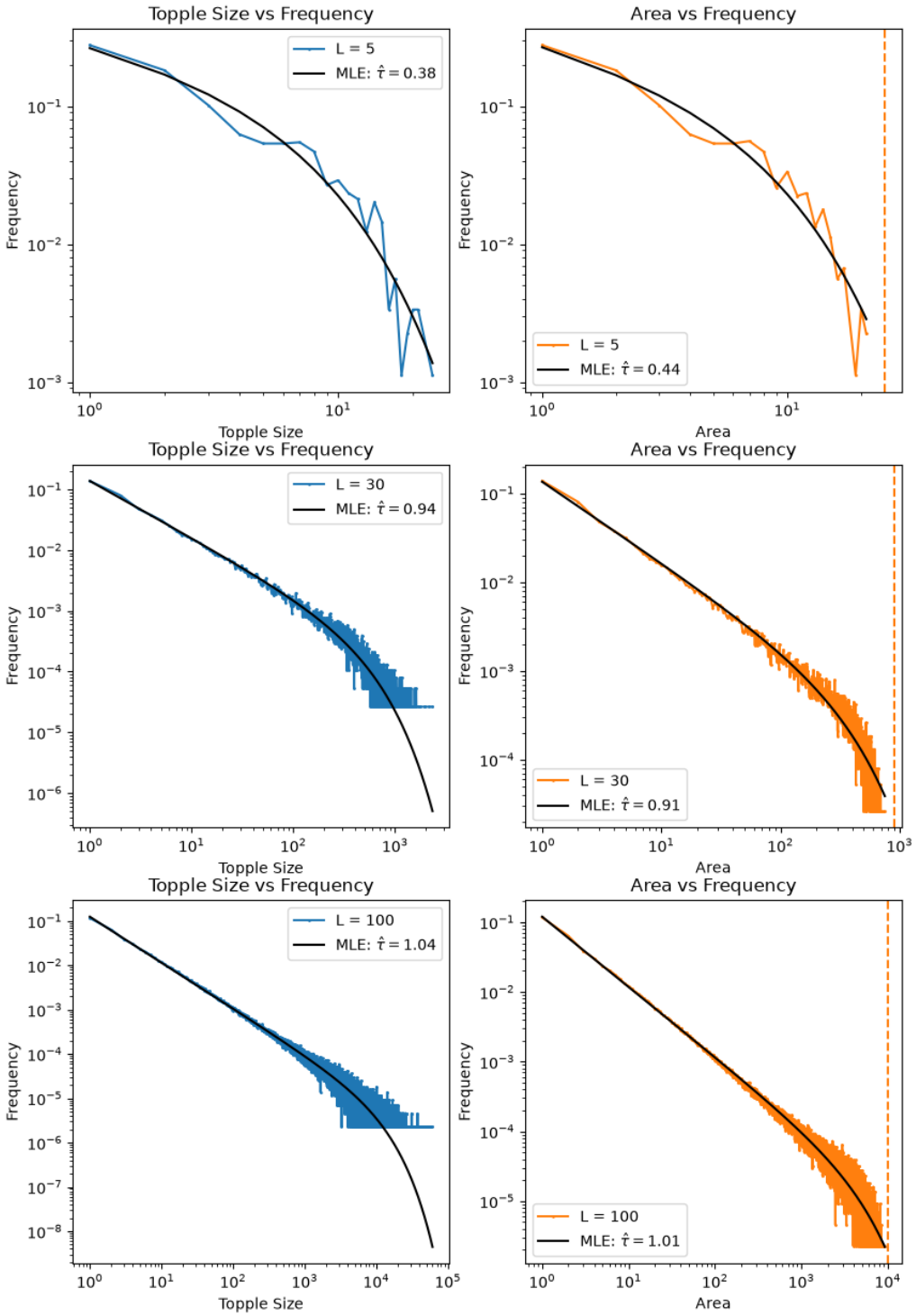


Figure 4.2: Fitted MLEs for the exponentially-truncated power law distribution against the observed data.

5

Conclusion

5.1 Reflection

Evidence of problem solving and creativity

Evidence that student has extended their knowledge

Shows initiative

Bibliography

[1] P. Bak, C. Tang, and K. Wiesenfeld. Self-organized criticality: An explanation of the 1/f noise. *Physical review letters*, 59(4):381, 1987.

[2] P. Bak, C. Tang, and K. Wiesenfeld. Self-organized criticality: An explanation of the 1/f noise. *Phys. Rev. Lett.*, 59:381–384, Jul 1987.

[3] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on scientific computing*, 16(5):1190–1208, 1995.

[4] D. Dhar. Self-organized critical state of sandpile automaton models. *Physical review letters*, 64(14):1613, 1990.

[5] A. A. Járai. Sandpile models. *Probability Surveys*, 15(none):243 – 306, 2018.

[6] S. K. Lam, A. Pitrou, and S. Seibert. Numba: A llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, pages 7:1–7:6, New York, NY, USA, 2015. ACM.